

Usar un giroscopio personalizado

Configura un giroscopio que el parser de YAGSL no crea automáticamente, utilizando un AHRS de Studica, como ejemplo.

Si tu giroscopio es uno de los [tipos compatibles](#) (Pigeon 2, Canandgyro o NavX3-CAN), no necesitas seguir esta guía. Simplemente configura `gyro.type` de la forma habitual en `swervedrive.json`.

Debes utilizar un giroscopio `custom` cuando tu hardware no sea uno de los dispositivos que el parser puede configurar automáticamente. El caso mas comun es el antiguo [Studica AHRS](#), conectando al roboRIO mediante SPI, I2C o USB serial (el clasico NavX2). Actualmente, solo el NavX3 basando en CAN es compatible de forma directa, por lo que un AHRS conectado al puerto MXP/SPI necesita este método alternativo. El mismo procedimiento también funciona con cualquier otra IMU, una fuente de orientación simulada o compuesta, o una orientación obtenida mediante la combinación de varios sensores.

1. Configura `gyro.type` como `custom`

En `swervedrive.json`, establece el `type` del giroscopio como `custom`. Los campos `id/canbus` no se utilizan con un giroscopio personalizado, así que déjalos como `0/""`, respectivamente. Además, y este es un detalle que suele causar problemas, `gyroAxis` y `gyroInvert` también **se ignoran**. Cuando el parser detecta `custom`, omite por completo la configuración automática del giroscopio.

```
JSON
{
  "gyro": { "type": "custom", "id": 0, "canbus": "" },
  "gyroAxis": "yaw",
  "gyroInvert": false,
  "modules": ["frontleft.json", "frontright.json", "backleft.json",
    "backright.json"]
}
```

2. Crea el giroscopio y proporcionar manualmente

Como el parser nunca llama a `SwerveDriveConfig.withGyro()` cuando se utiliza un giroscopio `custom`, debes llamarlo tú mismo el el `SwerveDriveConfig` que crees **antes** de

pasarlo a `SwerveParser.createSwerveDrive(...)` Todo lo demás, como los módulos, controladores de motor y encoders, seguirá configurándose normalmente a partir del resto del archivo JSON.

```
Java
import com.studica.frc.AHRS;
import static edu.wpi.first.units.Units.Degrees;

// El puerto SPI del MXP integrado es una conexion habitual para montar un AHRS en
// el roboRIO.
private final AHRS gyro = new AHRS(AHRS.NavXComType.kMXP_SPI);

public SwerveDriveSubsystem() {
    SwerveDriveConfig cfg = new SwerveDriveConfig()
        .withSubsystem(this)
        .withTelemetry(TelemetryVerbosity.HIGH)
        .withGyro(() -> Degrees.of(gyro.getAngle()));

    drive = new SwerveParser(new File(Filesystem.getDeployDirectory(),
        "swerve/base"))
        .createSwerveDrive(cfg);
}
```

Si olvidas llamar a `withGyro()` antes de `createSwerveDrive()` YAMS no tendrá ninguna fuente de orientación con la que crear el sistema de conducción. En ese caso, al iniciar el programa debes esperar una `NoSuchElementException/RuntimeException` alguna configuración alternativa.

3. Gestiona manualmente la inversion

Como `gyroInvert` se ignora, cualquier inversión de realizarse directamente dentro del `Supplier<Angle>` que pasas a `withGyro()`. Después, llama tu mismo a `withGyroInverted()` para mantener esta configuración sincronizada con culturizar otra parte de YAMS que consulte ese indicador.

```
Java
.withGyro(() -> Degrees.of(-gyro.getAngle())) // El AHRS usa CW+; YAGSL/YAMS
esperan CCW+
.withGyroInverted(false)
```

4. Verifica que funcione correctamente

Realiza la comprobación habitual del giroscopio para verificar que utiliza la [convención CCW+](#). [Mechanisms/swerve/gyro](#) en NetworkTables debería seguir mostrando datos y comportándose exactamente igual que un giroscopio creado automáticamente por el parser. Desde el punto de vista de YAMS, un [Supplier<Angle>](#) sigue siendo un [Supplier<Angle>](#) independientemente de donde proceden sus datos. Si el valor del ángulo aumenta cuando el robot gira en sentido antihorario, la configuración es correcta. Si no es así, invierte el signo de valor proporcionado por el Supplier, como se muestra en el paso 3, en lugar de cambiar [gyroInvert](#), ya que, como se explicó anteriormente, ese campo se ignora.

Opcional: velocidad angular para la corrección de skew

Si también quieres utilizar la velocidad angular del giroscopio para corregir el skew del sistema swerve (consulta [Swerve Drive Drift](#)), con un giroscopio [custom](#) debes proporcionar manualmente la velocidad angular de la misma manera que proporcionaste la orientación. El parser tampoco la crea automáticamente por ti.

Java

```
.withGyroVelocity(() -> DegreesPerSecond.of(-gyro.getRate()))  
.withGyroAngularVelocityScaleFactor(1.0)
```

[AHRS.getRate\(\)](#) y [getAngle\(\)](#) utilizan la misma convención de signos en el hardware de Studica: ambos consideran positivo el giro en sentido horario (CW+). Por lo tanto, si invertiste el signo de uno de ellos para obtener la convención CCW+, debes invertir también el signo del otro. De lo contrario, la corrección de skew actuará en sentido contrario a la orientación que debería estar corrigiendo.